

Complete.PIF.Analysis.partTwo.cluster_documentation

Fred Cudhea

08 May, 2025

Documentation for Complete.PIF.Analysis.partTwo.cluster.r. This script runs after "part one". After you've generated a distribution of PIFs/attributable mortality/incidence for each subgroup/dietary factor/disease, you run this to generate summary statistics (median, uncertainty intervals) of the output generated in part one.

First, read in relevant files, including the PIFs/attributable mortality sims file (allmort), observed mortality/incidence sims file (total mort), and observed population sims file (pop.draws)

```
##Complete.PAF.Analysis.partTwo.redone.r

allmort<-read.csv(file=paste(output_location, diet_pattern, "/all.incidence.draws_", year.vec.string, "_", covar.vec.string,
allmort<-allmort[order(allmort$riskfactor, allmort$outcome, allmort$female, allmort$age, allmort$race),]

#setwd("C:\\Users\\fcudhe01\\Box\\Lasting_aim_3\\data\\in\\example_cra")
#setwd(paste("C:\\users\\", Sys.info()[7], "\\Box\\Lasting_aim_3\\model development\\data_new\\in\\TEMP", sep=""))

setwd(file_location)

#totalmort<-read.csv("observed.cancer.incidence.draws.csv")
totalmort<-read.csv("observed.cancer.mortality.draws.csv")

pop.draws<-read.csv("observed.pop.draws.csv")
#pop.draws<-read.csv("population.sims.csv")
```

For pop.draws, rename sim variables (specifically change "X123", to "Y123", etc...). This will come in handy when we later merge with files that use "X123" naming scheme.

```
names(pop.draws)<-gsub("X", "Y", names(pop.draws))
#pop.draws$age<-0
#pop.draws$age[pop.draws$Age==1]<-25
#pop.draws$age[pop.draws$Age==2]<-35
#pop.draws$age[pop.draws$Age==3]<-45
#pop.draws$age[pop.draws$Age==4]<-55
#pop.draws$age[pop.draws$Age==5]<-65
#pop.draws$age[pop.draws$Age==6]<-75
```

Recode demographic variables in totalmort.

```
#totalmort<-totalmort[order(totalmort$disease, totalmort$female, totalmort$agecat, totalmort$race),]
totalmort<-totalmort[order(totalmort$disease, totalmort$subgroup),]
totalmort$age<-0
totalmort$age[totalmort$Age_label=="20-34"]<-25
totalmort$age[totalmort$Age_label=="35-44"]<-35
totalmort$age[totalmort$Age_label=="45-54"]<-45
totalmort$age[totalmort$Age_label=="55-64"]<-55
totalmort$age[totalmort$Age_label=="65-74"]<-65
totalmort$age[totalmort$Age_label=="75+"]<-75

totalmort$female[totalmort$Sex==1]<-1
totalmort$female[totalmort$Sex==2]<-0

totalmort$race[totalmort$Race_label=="NHW"]<-1
totalmort$race[totalmort$Race_label=="NHB"]<-2
totalmort$race[totalmort$Race_label=="HIS"]<-3
totalmort$race[totalmort$Race_label=="OTH"]<-4

#raw.file$Sex<-factor(raw.file$Sex)
```

Recall that, for the purposes of calculating the PIFs, we treat each outcome/pathway as a distinct outcome (direct effect, mediated by BMI, and mediated by SBP). But for purposes of collapsing outcomes (e.g. calculating joint PIFs), it's better to have the outcomes and pathway separated into two different variables. Below is code do that.

```
totalmort[!grepl("_", totalmort$disease, fixed=TRUE),]$disease<-paste(totalmort[!grepl("_", totalmort$disease, fixed=TRUE),]$
totalmort <- totalmort %>% separate(col=disease, into=c("outcome", "pathway"), sep="_")
totalmort<-totalmort[totalmort$pathway=="direct",]
totalmort<-subset(totalmort, outcome %in% diseases.vec, select=-c(pathway))
#totalmort<-subset(totalmort, outcome %in% diseases, select=-c(pathway))

#totalmort$outcome<-totalmort$disease
#totalmort$outcome_all<-gsub(pattern="_medBMI|_medSBP", replacement="", x=totalmort$outcome)

#totalmort$disease[grepl("_", totalmort$disease, fixed=TRUE) ]
```

Recall, we also have three broader outcome categories of interest (denoted as disease type 1, diseases type 2, disease type 3), and have mapped each outcome to each category (for all three) in LASTING_cluster_w_masterinput.r, and that mapping is stored in disease.table. Code below uses that mapping to create variables for the three disease type categories.

```
#totalmort$disease_type<-disease.type.vec[match(totalmort$outcome, disease.table)]
totalmort$disease_type<-disease.type.vec[match(totalmort$outcome[totalmort$outcome %in% diseases.vec], disease.table)]
totalmort$disease_type1<-disease.type1.vec[match(totalmort$outcome[totalmort$outcome %in% diseases.vec], disease.table)]
totalmort$disease_type2<-disease.type2.vec[match(totalmort$outcome[totalmort$outcome %in% diseases.vec], disease.table)]
totalmort$disease_type3<-disease.type3.vec[match(totalmort$outcome[totalmort$outcome %in% diseases.vec], disease.table)]
```

Get rid of unused demo vars (we'll be using the new recoded ones we created earlier).

```
totalmort<-totalmort[,~which(names(totalmort) %in% c("X", "disease", "Sex", "age_gp", "race_gp", "agecat"))]

# allmort$disease_type2<-disease.type2.vec[match(allmort$outcome[allmort$outcome %in% diseases.vec], disease.table)]
# allmort$disease_type3<-disease.type3.vec[match(allmort$outcome[allmort$outcome %in% diseases.vec], disease.table)]
```

Read in PIF sims file.

```
#setwd("C:\\Users\\fcdudhe01\\Box\\Lasting_aim_3\\data\\out\\example_cra")
#setwd(paste("C:\\users\\", Sys.info()[7], "\\Box\\Lasting_aim_3\\model development\\data_new\\out", sep=""))
pif<-read.csv(file=paste(output_location, diet_pattern, "/all.PIFs_", year.vec.string, "_", covar.vec.string, "_", diet_patte
```

Read in script that defines functions for summing up by demographic variables.

```
#setwd("C:\\Users\\fcdudhe01\\Box Sync\\new Life\\LASTING\\Code")
source(paste0(code_location, "/sum.by.strata.redone.r"))
```

Get summary stats of attributable mortality/incidence for each subgroup/dietary-factor+pathway/outcome. Rename certain variables of resulting variables for clarity.

```
##summary stats of mort
sum.stats.attr.mort.PIF<-sum.stats.maker(allmort=allmort)
sum.stats.PIF<-sum.stats.maker(allmort=pif)

names(sum.stats.attr.mort.PIF)[(1+length(covar.vec)):(6+length(covar.vec))]<-c("mean (food)", "se (food)", "sd (food)", "mean
names(sum.stats.PIF)[(1+length(covar.vec)):(6+length(covar.vec))]<-c("mean (food)", "se (food)", "sd (food)", "mean (food, co
```

read in original exposure file, strictly for the purpose of calculating "I", the number of subgroups. Note this is only valid assuming you subgroups are defined by age, sex and race. If that changes, this code will need to change as well. (Why do it this way rather than using mega input file, or output files? not sure. The code most likely predates the mega-input file, and maybe have even predated the creation of outputs (as in, pasted from part one for convenience)).

```
#setwd("C:\\Users\\fcudhe01\\Box\\lasting_aim_3\\data\\in\\example_cra")
#setwd(paste("C:\\users\\", Sys.info()[7], "\\Box\\lasting_aim_3\\model development\\data_new\\in\\TEMP", sep=""))

#mort.dat<-read.csv(file=paste("NHANES_Incidence_merged_", year.vec.string, "_", covar.vec.string, "_updated_", version.date,
mort.dat<-read.csv(file=paste("nhanes1518_agesexrace_merged_", version.date, "_FINAL.csv", sep=""), header=T) #Read CSV file
mort.dat<-mort.dat[order(mort.dat$diet),]

if(identical(covar.vec, c("Age", "Sex", "Race"))){
  l<-length(table(mort.dat$agecat))*length(table(mort.dat$female))*length(table(mort.dat$race))
}

#setwd("C:\\Users\\fcudhe01\\Box\\lasting_aim_3\\data\\out\\example_cra")
#setwd(paste("C:\\users\\", Sys.info()[7], "\\Box\\lasting_aim_3\\model development\\data_new\\out", sep=""))
```

Now we will calculate “reverse engineered PIFs”, meaning we are going to back-calculate PIFs from taking the attributable mortality/incidence and dividing my total number of deaths/incidence. This is how we calculate PIFs for **broader** subgroups.

First, we merge “allmort”, “totalmort” and “pop.draws” into one file, and identify the column index where part starts in this new dataframe.

```
#####Now let's calculate reverse-engineered PAFs for each draw
merged<-reduce(list(allmort, totalmort, pop.draws), merge)

allmort.starting.point<-which(names(merged)==“V1”)
allmort.ending.point<-which(names(merged)==paste(“V”, nsim1, sep=“”))
#allmort.ending.point<-which(names(merged)==“riskfactor”)-1
totalmort.starting.point<-which(names(merged)==“X1”)
totalmort.ending.point<-which(names(merged)==paste(“X”, nsim1, sep=“”))
#totalmort.ending.point<-dim(merged)[2]
pop.starting.point<-which(names(merged)==“Y1”)
pop.ending.point<-which(names(merged)==paste(“Y”, nsim1, sep=“”))
```

Create a new matrix with “reverse-engineered” PIFs calculated by dividing “allmort” values (attributable mort) by “totalmort” values (observed mort). **Order it**. If any NAs (from observed mort values being zero for whatever reason), change to 0. Save.

```
RE.PIFs<-cbind(merged[,which(names(merged) %in% c(“age”, “female”, “race”, “outcome”, “pathway”, “disease_type”,
“disease_type1”, “disease_type2”, “disease_type3”,
“riskfactor”))],
merged[,allmort.starting.point:allmort.ending.point]/merged[,totalmort.starting.point:totalmort.ending.point])

# check - Looks good
# RE.PIFs %>% arrange(outcome, riskfactor) %>% View()

RE.PIFs<-RE.PIFs[order(RE.PIFs$riskfactor, RE.PIFs$disease_type, RE.PIFs$outcome, RE.PIFs$female, RE.PIFs$age, RE.PIFs$race),
RE.PIFs[is.na(RE.PIFs)]<-0
write.csv(x=RE.PIFs, file=paste(output_location, diet_pattern, “/all.mort.draws_RE.PIFs_”, year.vec.string, “_”, covar.vec.st
```

Next, we use the same idea to get “**standardized** mortality”, as in the number of people saved (if at the counterfactual), per 100,000 people. So now, we take attributable mort, divide by total population, then multiply by 100,000.

```

standardized.mort.info<-merged[,which(names(merged) %in% c("age", "female", "race", "outcome", "pathway", "disease_type",
"disease_type1", "disease_type2", "disease_type3",
"riskfactor"))]

standardized.mort<-cbind(standardized.mort.info,
merged[,allmort.starting.point:allmort.ending.point]/merged[,pop.starting.point:pop.ending.point]*100000)
standardized.mort<-standardized.mort[order(standardized.mort$riskfactor, standardized.mort$outcome, standardized.mort$disease
standardized.mort$female, standardized.mort$age,
standardized.mort$race),]
standardized.mort[is.na(standardized.mort)]<-0
write.csv(x=standardized.mort, file=paste(output_location, diet_pattern, "/standardized.mort_", year.vec.string, "_", covar.v

```

Now we have a distribution of “reverse-engineered” PIFs and events prevented per 100,000 for all subgroups. Now we take those and [get summary stats](#) (and edit var names for standardized.mort.sum.stats file for clarity).

```

RE.PIFs.sum.stats<-sum.stats.maker.RE.PAFs(RE.PIFs)
standardized.mort.sum.stats<-sum.stats.maker(standardized.mort)
names(standardized.mort.sum.stats)[(length(names(standardized.mort.sum.stats))-4):length(names(standardized.mort.sum.stats))]
paste("mort.per.100k.",names(standardized.mort.sum.stats)[(length(names(standardized.mort.sum.stats))-4):length(names(stand
#####

```

Merge the summary stats files for attributable mort/incidence, “reverse-engineered” PIFs, and standardized attributable mort/incidence, and save it. Also, save the summary stats file for the actual PIFs (not “reverse-engineered”) we made [earlier](#).

```

sum.stats.attr.mort.merged<-reduce(list(sum.stats.attr.mort.PIF, RE.PIFs.sum.stats, standardized.mort.sum.stats), merge)

write.csv(x=sum.stats.attr.mort.merged, file=paste(output_location, diet_pattern, "/summarystats_attributable_USmortality_",
write.csv(x=sum.stats.PIF, file=paste(output_location, diet_pattern, "/summarystats_PIFs_", covar.vec.string, "_",year.vec.string,
"

# for(i in 1:length(sensitivity.vector))
# {
#   write.csv(x=create.sensitivity.output(output_file=sum.stats.attr.mort.merged, prop=sensitivity.vector[i]),
#             file=paste(output_location, diet_pattern, paste("/sensitivity_", sensitivity.vector[j], sep=""),
#             "/summarystats_attributable_USmortality_", covar.vec.string, "_",year.vec.string, "_", diet_pattern,
#             "_sensitivity_", sensitivity.vector[j], ".csv", sep=""),
#             row.names=FALSE)
# }

# sum.stats.total.attr.mort.PIF<-sum.stats.attr.mort.merged[grep(pattern="total", sum.stats.attr.mort.merged$outcome),]
# sum.stats.total.PIF<-sum.stats.PIF[grep(pattern="total", sum.stats.PIF$outcome),]
#
# write.csv(x=sum.stats.total.attr.mort.PIF, file=paste("summarystats_attributable_USmortality_total_", covar.vec.string, "_",
#             row.names=FALSE)
# write.csv(x=sum.stats.total.PIF, file=paste("summarystats_PAFs_total_", covar.vec.string, "_",year.vec.string, ".csv", sep=
#             row.names=FALSE)
#####

```

Ok, now that we’ve done that. Let’s get to making output that you’ll actually be reporting in your paper. That is, [summary statistics for broader subgroups](#) (results aby age, results by sex, results by dease type, etc... as opposed results by each age/sex/race/outcome/etc..). We will create results for all possible combinations that could be of interest here but you will choose what to report based on what your research focus is.

First we put all strata of interest into a vector called “strata”. Then we loop through the length of that vector. For each iteration i of that loop, we extract all combinations of that vector with i element using combn function, convert the reslting matrix to a list, and append those list elements to “strata.combos” list which will have an element for each possible strata combination.

```
strata.combos<-list()
strata<-c("age", "female", "race", "outcome", "pathway", "disease_type", "disease_type1", "disease_type2", "disease_type3")
#strata<-names(allmort)[c(1:(which(names(allmort))=="mean..food.")-1), which(names(allmort) %in% c("outcome_all", "disease_type1", "disease_type2", "disease_type3"))]
##order should match with covar.vec
for(i in 1:length(strata))
{
  x<-combn(strata, i)
  combos.for.i<-split(x, rep(1:ncol(x), each = nrow(x))) #https://stackoverflow.com/questions/6819804/how-to-convert-a-matrix-to-a-list
  strata.combos<-c(strata.combos, combos.for.i)
}
```

Not all combinations are of interest. We want to, at the very least, have outcome pathway or a disease type as a strata (Results are event specific after all. We aren't reporting all preventable events by age when the types of events are not even consistent by outcome). Also, not all the strata are mutually exclusive. It's redundant to stratify by disease type 1 if you're already stratifying by outcome so we don't need any strata combos that has both outcome and disease type 1, for example. We want to filter out unnecessary strata combos, which is what the code below does (we loop through the list of strata combos and identify the index the strata combos that qualify as of interest).

An astute reader might note that stratifying just by "pathway" and not outcome/disease type will mix up incidence (from cancer) and mortality (from CMDs). Good point! You got me!

```
##interested in only ones where there is at least one var that is outcome, pathway, disease, and not interested if outcome and pathway
strata.combos.of.interest.boolean<-c()
for(i in 1:length(strata.combos))
{
  disease_overlap_count<-sum("outcome" %in% strata.combos[[i]], "disease_type" %in% strata.combos[[i]], "disease_type1" %in% strata.combos[[i]], "disease_type2" %in% strata.combos[[i]])
  strata.combos.of.interest.boolean[i]<-any(strata.combos[[i]] %in% c("outcome", "pathway", "disease_type", "disease_type1", "disease_type2"))
  #strata.combos.of.interest.boolean[i]<-any(strata.combos[[i]] %in% c("outcome", "pathway", "disease_type", "disease_type1", "disease_type2"))
}
strata.combos.of.interest.index<-which(strata.combos.of.interest.boolean)

# check
print(strata.combos.of.interest.index)
```

Having identifying the index for each strata combo of interest, we can loop through them and apply the Sum.by.strata command which takes as inputs the attributable mort sims, total mort sims, total pop sims, and the stratas of interest, and uses that to calculate 4 output files of interest: 1: summary stats of attributable mort/incidence + "reverse engineered" PIFs + standardized preventable mort/incidence by specified strata, 2: entire simulation of attributable mort/incidence by specified strata, 3: entire simulation of observed deaths by specified strata, 4: entire simulation of "reverse engineered" PIFs by specified strata.

```

# debug
# i = 264

for(i in strata.combos.of.interest.index)
{
  print(i)
  print(strata.combos[[i]])
  print("")

  sum.stats.byX.attr.mort<-Sum.by.strata(allmort=allmort, totalmort=totalmort, pop=pop.draws, covar=strata.combos[[i]])

  write.csv(x=sum.stats.byX.attr.mort[[1]], file=paste(output_location, diet_pattern, "/summarystats_attributable_USmortality",
    row.names=FALSE)
  write.csv(x=sum.stats.byX.attr.mort[[2]], file=paste(output_location, diet_pattern, "/attributable_USmortality_draws_", "by",
    row.names=FALSE)
  write.csv(x=sum.stats.byX.attr.mort[[3]], file=paste(output_location, diet_pattern, "/total_USmortality_draws_", "by_", pas
    row.names=FALSE)
  write.csv(x=sum.stats.byX.attr.mort[[4]], file=paste(output_location, diet_pattern, "/RE_PIFs_draws_", "by_", paste(strata.
    row.names=FALSE)

  # for(j in 1:length(sensitivity.vector))
  # {
  #   write.csv(x=create.sensitivity.output(output_file=sum.stats.byX.attr.mort[[1]], prop=sensitivity.vector[j]),
  #     file=paste(output_location, diet_pattern, paste("/sensitivity_", sensitivity.vector[j], sep=""),
  #     "/summarystats_attributable_USmortality_", "by_",
  #     paste(strata.combos[[i]], sep=" ", collapse=" "), "_", year.vec.string, "_", diet_pattern,
  #     "_sensitivity_", sensitivity.vector[j], ".csv", sep=""),
  #     row.names=FALSE)
  # }

}

#don't need anymore
# sum.stats.byOverall.attr.mort<-Sum.by.overall(allmort=allmort, totalmort=totalmort, pop=pop.draws)
# write.csv(x=sum.stats.byOverall.attr.mort[[1]], file=paste("out/summarystats_attributable_USmortality_", "by_", "Overall",
#   row.names=FALSE)
# write.csv(x=sum.stats.byOverall.attr.mort[[2]], file=paste("out/attributable_USmortality_draws_", "by_", "Overall", "_", ye
#   row.names=FALSE)
# write.csv(x=sum.stats.byOverall.attr.mort[[3]], file=paste("out/total_USmortality_draws_", "by_", "Overall", "_", year.vec.
#   row.names=FALSE)
# write.csv(x=sum.stats.byOverall.attr.mort[[4]], file=paste("out/RE_PIFs_draws_", "by_", "Overall", "_", year.vec.string, ".
#   row.names=FALSE)

##tehn figureo ut functions for RE.PAFs by strata + draws for RE.PAFs and and total mort by strata (actually don't need sepe
##Then, instead of writing it out, go through all possible combiantoiois of strata combos ain a for looop and and calling the f

```